

INFORME # 2: METODOLOGÍA PROPUESTA

Proyecto: Clasificador y Organizador Automático de Fotografías por Tipo de Escena
Integrantes: Jefferson Saltos y Danilo Drouet
Curso: Procesamiento Digital de Imágenes (DIP)

2. Metodología propuesta

El desarrollo de este proyecto se rige bajo la metodología ágil Scrum, dividida estratégicamente en tres iteraciones o sprints. Tras haber definido formalmente la problemática y la estructura modular en el Sprint 1, este segundo informe se enfoca en el Sprint 2: Definición de la Metodología del Proyecto. Aquí se establece el marco técnico, el análisis de datos, el alcance, los requerimientos detallados, el diseño conceptual y la planificación requerida para garantizar la viabilidad y correcta ejecución de los tres módulos interconectados del sistema.

2.1 Análisis de los datos

Para garantizar el correcto funcionamiento del clasificador automático, es fundamental analizar la naturaleza y el flujo de los datos a través de cada componente. A continuación, se presenta la tabla con la especificación técnica de las entradas, los procesos y las salidas correspondientes a las diferentes fases de la arquitectura:

Componente / Fase	Datos de Entrada	Proceso / Análisis Técnico	Datos de Salida
Entrada General	Colecciones de imágenes variadas en formatos estándar (.jpg, .jpeg, .png) que abarcan múltiples tipos de escenas (paisajes, retratos, objetos y eventos).	Recopilación masiva de imágenes de fuentes de uso libre o archivos personales locales para alimentar el sistema.	Imágenes originales en bruto cargadas en memoria como matrices de píxeles.
Módulo #1: Detección	Matrices de píxeles correspondientes a las imágenes crudas	Inferencia visual mediante modelos preentrenados. El	Tensores numéricos (coordenadas de bounding boxes) y

Componente / Fase	Datos de Entrada	Proceso / Análisis Técnico	Datos de Salida
	ingresadas.	modelo analiza la geometría y patrones para localizar elementos clave de interés.	recortes vectoriales aislados de los objetos principales.
Módulo #2: Procesamiento y Análisis Visual	Imágenes originales completas y recortes de objetos generados por el Módulo #1.	Manipulación de canales de color (espacios RGB, HSV o LAB) para color dominante; evaluación de matrices de coocurrencia o gradientes para la textura; y aplicación de operadores matriciales para bordes y binarización.	4 tipos de insumos visuales optimizados: bordes crudos, bordes filtrados (sin ruido), máscaras binarias (ceros y unos) y objetos segmentados.
Módulo #3: Organización de Archivos	Insumos visuales procesados, variables de salida estructuradas y arreglos consolidados de metadatos.	Análisis lógico de las propiedades y metadatos extraídos para mapearlos dinámicamente en una estructura jerárquica física de almacenamiento.	Rutas físicas creadas en el directorio local, estructura jerárquica organizada de carpetas y reporte analítico final (TXT, JSON o PDF).

2.2 Alcance del proyecto

El alcance define los límites y compromisos de entrega para la solución tecnológica:

Lo que **INCLUYE** el proyecto (Dentro de alcance):

- Desarrollo de un script o herramienta de software modular en Python que automatice el flujo completo en una sola ejecución.
- Integración de modelos preentrenados (como YOLO, SSD, Faster R-CNN o SAM) para la detección y segmentación precisa de elementos.
- Implementación de algoritmos de PDI para la extracción de características (color, textura) y generación de 4 tipos de insumos visuales: bordes crudos, bordes filtrados (sin ruido), máscaras binarias y objetos segmentados.
- Clasificación dinámica y almacenamiento automatizado de archivos en una estructura jerárquica de carpetas locales.
- Generación automática de un reporte final de resultados en formato legible (TXT, JSON o PDF) con el resumen de la metadata extraída.

Lo que NO INCLUYE el proyecto (Fuera de alcance):

- El entrenamiento o ajuste fino (fine-tuning) de arquitecturas de Deep Learning desde cero (se usarán estrictamente modelos preentrenados).
- Desarrollo de una aplicación web en la nube o arquitectura basada en microservicios cliente-servidor (la herramienta será de ejecución local).
- Procesamiento de archivos de video en tiempo real.

2.3.1 Requerimientos

Los requerimientos del sistema dictan las capacidades funcionales específicas y las restricciones técnicas:

Requerimientos Funcionales (RF):

- **RF-01: Carga de Colecciones:** El sistema debe permitir al usuario especificar la ruta local de una carpeta que contenga la colección de imágenes de entrada.
- **RF-02: Inferencia Automatizada:** El sistema debe invocar un modelo preentrenado para identificar los objetos principales y extraer sus coordenadas (bounding boxes) sin intervención manual.
- **RF-03: Procesamiento de Bordes:** El sistema debe generar una versión de bordes crudos y una versión de bordes filtrados/sin ruido utilizando técnicas de PDI para cada imagen.
- **RF-04: Segmentación y Máscaras:** El sistema debe aislar los objetos detectados generando su correspondiente máscara binaria y la imagen del objeto segmentado sobre fondo neutro.
- **RF-05: Extracción de Atributos:** El sistema debe computar y registrar el color dominante, densidad de bordes y características de textura de cada insumo visual.
- **RF-06: Estructuración de Directorios:** El sistema debe crear automáticamente carpetas diferenciadas (ej: /originales, /bordes_crudos, /bordes_procesados, /mascaras, /recortes) y guardar los archivos donde corresponda.
- **RF-07: Reporte Resumen:** El sistema debe exportar un archivo de reporte con estadísticas generales de las imágenes procesadas y la metadata recolectada.

Requerimientos No Funcionales (RNF):

- **RNF-01: Modularidad:** El código fuente debe estar estrictamente dividido en los 3 módulos definidos en la arquitectura para facilitar su mantenimiento.
- **RNF-02: Compatibilidad de Software:** La herramienta debe ser desarrollada en Python 3.10 o superior, asegurando compatibilidad multiplataforma (Windows/Linux/macOS).
- **RNF-03: Eficiencia en Procesamiento:** El tiempo promedio de procesamiento y análisis de características por imagen individual no debe exceder un límite razonable en hardware estándar.
- **RNF-04: Robustez ante el Ruido:** El módulo de filtrado de bordes debe remover eficazmente el ruido de alta frecuencia visual antes de la extracción final.

2.3.2 Recursos a utilizar

Hardware:

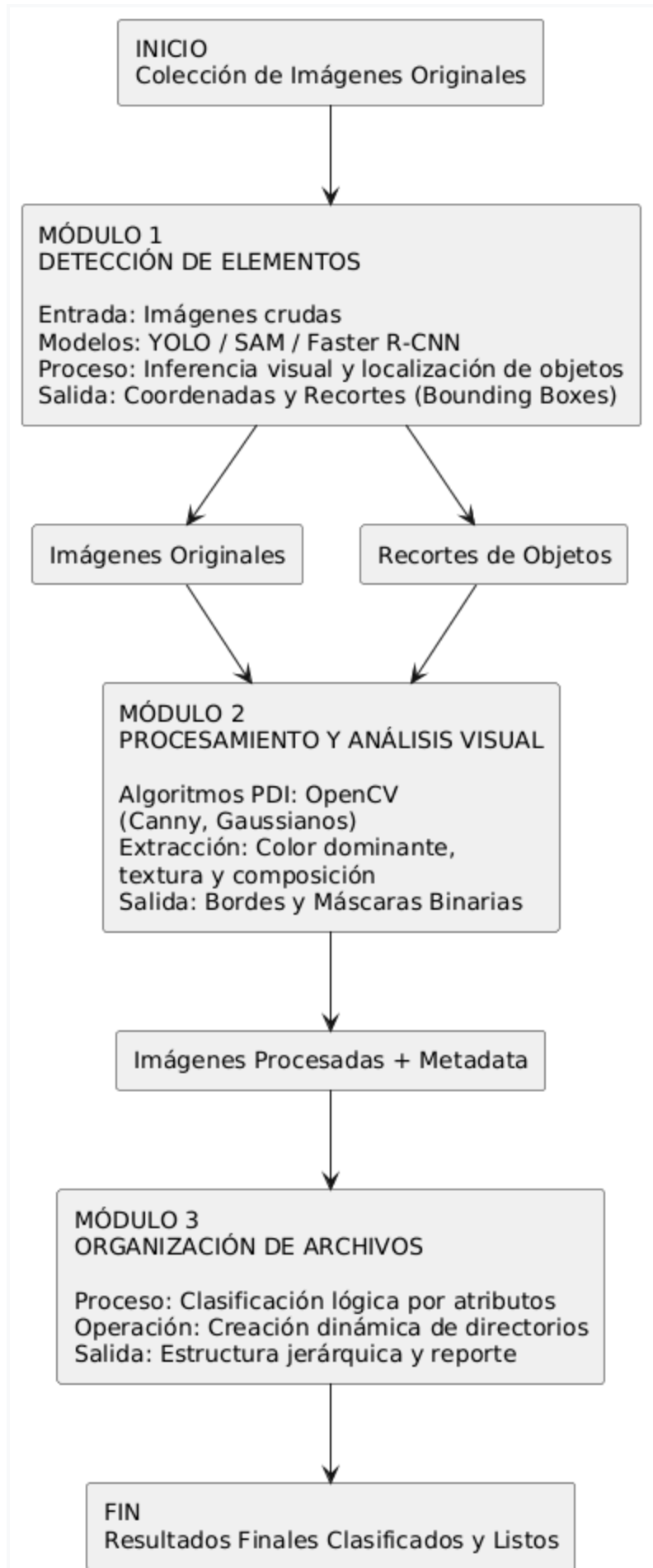
- Computadora personal con procesador multi-núcleo (Intel i5/AMD Ryzen 5 o superior).
- Memoria RAM mínima de 16 GB (para soportar la carga en memoria de imágenes y modelos de IA).
- Tarjeta gráfica dedicada (GPU NVIDIA compatible con CUDA recomendado) para acelerar la velocidad de inferencia.

Software y Bibliotecas (Stack Tecnológico):

- **Lenguaje de Programación:** Python 3.10+.
- **Visión por Computador y PDI:** OpenCV (para algoritmos de Canny, transformaciones de color, filtrado de ruido y umbralización).
- **Modelos de Inferencia y Deep Learning:** Frameworks PyTorch o TensorFlow junto con las librerías oficiales de Ultralytics (YOLO) o Meta AI para Segment Anything Model (SAM).
- **Machine Learning Auxiliar y Estadísticas:** Scikit-learn (para algoritmos de agrupamiento como K-Means aplicado a la extracción del color dominante).
- **Gestión del Entorno y Sistema:** Módulos nativos de Python (os, shutil, pathlib) para el manejo dinámico del sistema de archivos y directorios locales.

2.3.3 Diseño conceptual

El diseño conceptual describe de forma macro el ciclo de vida de los datos desde que ingresan al sistema hasta que se alojan ordenadamente en el almacenamiento local:



2.3.4 Bocetos y flujos de pantalla

Dado que la herramienta está diseñada primordialmente como un software automatizado de procesamiento para ingenieros de IA, la interfaz prioriza la claridad, la parametrización de tareas y la visualización de resultados estructurados.

Flujo de Pantallas:

- **Pantalla 1: Configuración e Inicio:** El usuario selecciona el directorio de imágenes, configura los parámetros de PDI y selecciona el modelo preentrenado.
- **Pantalla 2: Consola de Procesamiento Activo:** Muestra una barra de progreso, logs en tiempo real de las tareas y una vista previa del procesamiento.
- **Pantalla 3: Resumen Visual del Directorio y Reporte:** Despliega un árbol jerárquico con las carpetas resultantes creadas y un panel estadístico.

Esquema del Boceto 1: Pantalla de Configuración e Inicio (Wireframe)

CLASIFICADOR Y ORGANIZADOR AUTOMÁTICO DE FOTOGRAFÍAS - PROYECTO DIP	
[1] SELECCIÓN DE ENTRADA:	
Ruta de carpeta: [C:/Usuarios/IA_Engineer/Datasets/Fotos_Crudas] [..]	
[2] CONFIGURACIÓN DEL MODELO (MÓDULO 1):	
☒ YOLO v8 ☐ Segment Anything (SAM) ☐ Faster R-CNN	
[3] PARÁMETROS DE PROCESAMIENTO (MÓDULO 2):	
Umbral Canny Mín: [100] Umbral Canny Máx: [200]	
Reducción de Ruido: ☒ Filtro Gaussiano ☐ Filtro Mediana	
[4] DESTINO DE SALIDA:	
Ruta de Salida: [C:/Usuarios/IA_Engineer/Datasets/Resultados_DIP] [..]	
[INICIAR PROCESAMIENTO ►]	

2.4 Plan de implementación del proyecto

El plan de ejecución para el Sprint 3: Desarrollo, Análisis y Conclusiones se desglosa en un cronograma estructurado cronológicamente por etapas de desarrollo técnico continuo en lugar de semanas fijas:

Fase Temporal	Módulo / Actividad	Descripción de la Tarea Técnica	Entregable Esperado
Etap 1	Módulo #1: Detección y Configuración Inicial	Configuración del entorno unificado de desarrollo en Python. Carga del conjunto de imágenes de prueba e implementación de inferencia con el modelo seleccionado (YOLO/SAM) para la obtención y almacenamiento de recortes de objetos principales.	Script de inferencia completamente funcional y almacenamiento automatizado del primer lote de imágenes recortadas.
Etap 2	Módulo #2: Algoritmos de Procesamiento Digital de Imágenes	Programación e integración de operadores matriciales utilizando OpenCV para la extracción de bordes crudos y la remoción de ruido mediante filtros de suavizado. Ajuste y automatización de algoritmos de umbralización para generar las máscaras binarias del dataset.	Funciones y algoritmos de procesamiento de imágenes completamente integrados al flujo modular de trabajo.
Etap 3	Módulo #2 & #3: Extracción Avanzada y Lógica de Almacenamiento	Desarrollo de scripts de análisis estadístico empleando Scikit-learn para la identificación automática del color dominante y texturas. Codificación del motor lógico del Módulo #3 para la creación e indexación física en el sistema de archivos local de acuerdo a los atributos extraídos.	Pipeline unificado funcional de extremo a extremo con almacenamiento automático categorizado por directorios lógicos.
Etap 4	Pruebas Consolidadas, Cierre y Documentación Técnica	Pruebas masivas del flujo de automatización utilizando conjuntos de imágenes variadas y complejas. Evaluación cuantitativa y cualitativa de la consistencia del sistema de ordenamiento y generación automática del reporte definitivo. Redacción formal del informe Final.	Código fuente definitivo optimizado, libre de errores y documentación final integradora de los resultados del proyecto.